

FRD — Customer Retention Analytics Engine

Functional Requirements Document

1. Purpose and solution boundary

This document specifies the batch analytics workflow that produces a Power BI-ready churn-prediction export from the Telco Customer Churn dataset. It reflects the supplied Python implementation: Pandas cleaning, one-hot encoding, an 80/20 holdout split with

```
random_state=42 , and RandomForestClassifier(n_estimators=100,
random_state=42) .
```

The exported dataset is a holdout evaluation output, not a live customer-scoring service.

2. Input data contract

Source: Telco-Customer-Churn.csv (expected original size: 7,043 rows × 21 columns).

Required source columns: customerID , TotalCharges , Churn , MonthlyCharges , and the categorical service/contract/payment fields expected by the dataset.

Cleaning rules:

1. Drop customerID before modelling.
2. Convert TotalCharges with pd.to_numeric(errors='coerce') .
3. Drop rows with missing values after coercion.
4. One-hot encode categorical fields with pd.get_dummies(drop_first=True) .
5. Set y = Churn_Yes ; set X to all remaining encoded fields.

The known IBM Telco source normally loses rows whose blank TotalCharges values become missing; the exact post-cleaning row count must be checked against the delivered file at refresh time.

3. Model and output contract

Item	Required implementation
Split	<pre>train_test_split(X, y, test_size=0.2, random_state=42)</pre>
Model	<pre>RandomForestClassifier(n_estimators =100, random_state=42)</pre>

Prediction type	Binary class prediction: 0/1
Output file	<code>Churn_Predictions_For_Dashboard.csv</code>
Write method	<code>X_test_export.to_csv(..., index=False)</code>
Required outcome fields	<code>Actual_Churn</code> , <code>Predicted_Churn</code>
Required analysis field	<code>MonthlyCharges</code>
Feature fields	Remaining encoded model features, including dummy variables where present

The present supplied code does not export a churn probability, model feature importances, customer ID, training data, or threshold parameter. Do not describe a “high probability” threshold in the dashboard unless the Python export is enhanced with `predict_proba` and a documented threshold.

4. Functional requirements

ID	Requirement	Acceptance criteria
FR-01	Load source CSV.	Load succeeds; column names match required contract.
FR-02	Clean charges and missing records.	<code>TotalCharges</code> is decimal; rows with coercion failures/missing values are removed and counted.
FR-03	Encode features.	No non-numeric model inputs remain; <code>Churn_Yes</code> exists.
FR-04	Create reproducible holdout.	Test proportion is 20%; random state is 42.
FR-05	Train classifier.	Random Forest uses 100 estimators and random state 42.

FR-06	Create output.	Output contains one row per X_test record and no accidental index field.
FR-07	Retain labels and predictions.	Both fields are binary and non-null.
FR-08	Refresh Power BI.	Import types are valid; measures return reconciled values.
FR-09	Segment presentation.	Encoded fields receive friendly labels/data dictionary in Power Query or report labels.
FR-10	Support model validation.	Confusion-matrix measures compute TP, FP, FN, TN, accuracy, precision and recall safely.

5. Data-quality and validation checks

Run the following after every export:

- Assert no `Unnamed: 0` column exists.
- Confirm row count equals `len(X_test)`; expected value is derived from the cleaned source and split.
- Confirm `Actual_Churn` and `Predicted_Churn` contain only 0 and 1.
- Confirm `MonthlyCharges` is numeric and non-negative.
- Confirm the source ID is absent from the model export; this is expected from the supplied code.
- Recalculate the confusion matrix from the two output columns and compare it to `classification_report` / confusion-matrix results generated in Python.
- Recalculate count and revenue-at-risk outside Power BI once, then reconcile report measures.

6. Power BI functional design

Query name: `Churn Predictions`.

Power Query transformations:

- Promote headers, set outcome flags to whole number and charges to decimal number.
- Rename displayed dummy fields, for example `Contract_Month-to-month` → “Month-to-month contract flag”; do not convert a missing baseline category into an assumed label without documenting it.
- Add “Yes/No” display columns for 0/1 flags only where source semantics are known.
- Add a column dictionary in the report documentation: source field, friendly label, data type and interpretation.
- Keep the raw imported fields unchanged in the model-output table for auditability.

Report behaviour:

- Every visual must respect page slicers unless explicitly labelled otherwise.
- Interactions: selecting a segment filters KPIs and the detail table.
- Details table must include only fields available in the CSV. If no customer identifier is exported, it is an aggregated diagnostic table, not a contact list.
- At-risk filters are based on `Predicted_Churn = 1` ; actual churn filters are separate.

7. Non-functional requirements

Area	Requirement
Reproducibility	Pin input filename, model parameters and random state in the run record.
Auditability	Store export date/run ID externally or add it as a metadata field in a future enhancement.
Privacy	Do not expose customer identifiers or outreach data beyond role-appropriate access.
Performance	Current holdout export is small; report should load interactively without aggregation changes.
Maintainability	Centralise DAX measures and keep the raw column names documented.
Accessibility	Use explicit labels, adequate contrast, tab order, alt text and non-colour-only encoding.

8. Recommended controlled enhancement

For campaign priority and better ROI modelling, update the export to retain a governed `customerID` and add `Predicted_Churn_Probability = model.predict_proba(X_test)[: , 1]`. Document a business-approved threshold and score date. This is a future enhancement; it is not assumed in the current deliverable.

9. Data flow

Raw CRM CSV → cleaning and type coercion → one-hot encoding → holdout split → Random Forest training → holdout predictions + actual label → dashboard CSV → Power BI measures/segments → retention campaign decision → campaign outcome monitoring.